

SEARCHING

PROBLEM 2

QUESTION:

DS	Session	Searching	Question Information	Level 1	Challenge 2
Problem Description: King Alexander wants every chariot to line up for the start of her Winter-eve ball. He has asked you, Twilight Sparkle, to sort the horse chariots alphabetically but with royalty in front. Royal horses chariot have diamonds in their names. Constraints: 1 <= Names <= 100 Input Format: list of horse chariots numbering anywhere from 1 to 40 horse chariot, with 1 per line. The end of input will be marked with END on a line by itself. Names should be no longer than 100 characters in length, and to only contain letters and spaces. Output Format: Print the output in a separate lines contains Sort and list the horse chariots alphabetically in ascending order ('A' "first"), ignoring case. However, any horse chariot with a diamond in their name must be placed at the "top" of the list (before the "A's" start) in the diamond order given in the Discussion section below. Explanation: The Pony ranking (in Ascending order) for gemstones is a follows: Lapis, Topaz, Tourmaline, Sapphire, Peridot, Ruby, Pearl, Emerald, Diamond, Aquamarine, Amethyst, Garnet. you are guaranteed that you will not have to deal with any gemstones not listed above. If multiple gems are listed in the same name, sort by whichever gem has "highest priority" (e.g. a pony named 'Garnet Lapis' would be listed before a pony named 'Topaz Sapphire,' because Lapis has the highest priority). In the case of equal ranking on gemstones, output in alpha order by total name (not just the gemstones), only sort by highest priority gemstones, after that by alpha. (Example, given the names: Lapis Topaz and Amethyst Lapis, they should be printed in order as: Amethyst Lapis then Lapis Topaz. That is because both names have the highest ranked gemstone (Lapis), so we simply sort them in ascending order alphabetically after their ranking has been established. We would not list Lapis Topaz before Amethyst Lapis because Topaz has a higher ranking over Amethyst. Stop comparing gemstone ranking after determine the highest rank of the gems in the name.) You are guaranteed that there will be no ties in priority for gemstone ranking. You are also guaranteed that you will not encounter hyphenated names like Ruby-Sue. However, if a gemstone name happened to be found as part of a name (like Rubyanne) you can safely treat that as just another name to alphabetise. Gemstone names have to stand on their own					

(separated by spaces, or the entirety of the name) to be treated royally.

Logical Test Cases

Test Case 1

INPUT (STDIN)

Buttershy
Orangejack
Rubyanne
Rainbow Dash
Rarity
Pinkie Pie
Emerald Sunshine
Spike the Dragon
Misty Sapphire
END

EXPECTED OUTPUT

Misty Sapphire
Emerald Sunshine
Buttershy
Orangejack
Pinkie Pie
Rainbow Dash
Rarity
Rubyanne
Spike the Dragon

Test Case 2

INPUT (STDIN)

Granny Smith
Discord
Spitfire
Granny Smith
Photo Finish
Pearl Peridot
Granny Smith
Princess Pearl
Soarine
END

EXPECTED OUTPUT

Pearl Peridot
Princess Pearl
Discord
Granny Smith
Granny Smith
Granny Smith
Photo Finish
Soarine
Spitfire

Mandatory Test Cases

Test Case 1

KEYWORD

char *gems[]=
{ "NONE", "Garnet", "Amethyst", "Aquamarine", "Diamond", "Emerald", "Pearl", "Ruby", "Peridot", "Sapphire", "Tourmaline", "Topaz", "Lapis", 0};

Test Case 2

KEYWORD

char ponies[MAXP][BUFLN];

Test Case 3

KEYWORD

strcmp(ponies[a],ponies[b])>0;

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

530

Test Case 3

NLOC

60

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>

const char* GEM_TIERS[] = {
    "NONE", "Garnet", "Amethyst", "Aquamarine", "Diamond", "Emerald",
    "Pearl", "Ruby", "Peridot", "Sapphire", "Tourmaline", "Topaz", "Lapis", NULL
};

int calculate_gem_priority(const char* title) {
    int highest = 0;
    for (int idx = 1; GEM_TIERS[idx] != NULL; idx++) {
        if (strstr(title, GEM_TIERS[idx])) {
            if (idx > highest) highest = idx;
        }
    }
    return highest;
}

int main() {
    char entries[100][100];
    int total_entries = 0;

    while (fgets(entries[total_entries], 100, stdin)) {
        entries[total_entries][strcspn(entries[total_entries], "\r\n")] = '\0';
        if (strcmp(entries[total_entries], "END") == 0) break;
        total_entries++;
    }

    for (int i = 0; i < total_entries - 1; i++) {
        for (int j = i + 1; j < total_entries; j++) {
            int score_i = calculate_gem_priority(entries[i]);
            int score_j = calculate_gem_priority(entries[j]);

            int should_swap = 0;
            if (score_i != score_j) {
                if (score_i < score_j) should_swap = 1;
            } else {
                if (strcmp(entries[i], entries[j]) > 0) should_swap = 1;
            }

            if (should_swap) {
                char temp[100];
                strcpy(temp, entries[i]);
                strcpy(entries[i], entries[j]);
                strcpy(entries[j], temp);
            }
        }
    }

    for (int k = 0; k < total_entries; k++) {
        printf("%s\n", entries[k]);
    }
    return 0;
}
```

OUTPUT:

```
Buttershy
Orangejack
Discord
Spitfire
Rainbow Dash
Misty Sapphire
Pearl Peridot
Emerald Sunshine
END
Misty Sapphire
Pearl Peridot
Emerald Sunshine
Buttershy
Discord
Orangejack
Rainbow Dash
Spitfire
```

```
Granny Smith
Discord
Spitfire
Granny Smith
Photo Finish
Pearl Peridot
Granny Smith
Princess Pearl
Soarine
END
Pearl Peridot
Princess Pearl
Discord
Granny Smith
Granny Smith
Granny Smith
Photo Finish
Soarine
Spitfire
```